

Logistics & GUI — Factory Method + Abstract Factory (Java)

Assignment 2 for **ShP-2216 — Software Design Patterns**, Astana IT University. A single console application combines two creational patterns:

- **Factory Method** selects road or sea delivery (`Truck` / `Ship`).
- **Abstract Factory** selects a matching UI component family (`Windows` / `macOS` button and checkbox).

Both parts run together in one program, driven by two independent runtime choices.

Project purpose

The application simulates a logistics company that can deliver cargo by road or by sea, while its console "UI" can be rendered in a Windows or a macOS style. The two choices are completely independent: changing the platform never affects which transport is used, and vice versa.

Package structure

src/com/example/logistics/	Factory Method (Part A)
├ Transport.java	Product interface
├ Truck.java	Concrete product (road)
├ Ship.java	Concrete product (sea)
├ Logistics.java	Abstract creator (shared planDelivery)
├ RoadLogistics.java	Concrete creator -> Truck
└ SeaLogistics.java	Concrete creator -> Ship
src/com/example/gui/	Abstract Factory (Part B)
├ Button.java	Abstract product
├ Checkbox.java	Abstract product
├ WindowsButton.java	\
├ WindowsCheckbox.java	\ Windows family
├ MacOSButton.java	/
├ MacOSCheckbox.java	/ macOS family
├ GUIFactory.java	Abstract factory
├ WindowsFactory.java	Concrete factory (Windows family)
└ MacOSFactory.java	Concrete factory (macOS family)
src/com/example/app/	Client & startup
├ DeliveryMode.java	ROAD / SEA
├ UIPlatform.java	WINDOWS / MACOS
├ DeliveryApplication.java	Client: uses GUIFactory + Logistics
└ Main.java	Reads input, validates, wires everything
uml/	
├ factory_method.dot / .png	Factory Method class diagram
└ abstract_factory.dot / .png	Abstract Factory class diagram

Prerequisites

- JDK 17 (verify with `java -version` / `javac -version`)
- Any terminal, or IntelliJ IDEA

Build and run

Command line

```
cd src
javac com/example/logistics/*.java com/example/gui/*.java com/example/app/*.java -d ../out
java -cp ../out com.example.app.Main ROAD WINDOWS
```

If no arguments are given, the program prompts for them interactively:

```
java -cp ../out com.example.app.Main
Delivery mode (ROAD/SEA): SEA
UI platform (WINDOWS/MACOS): MacOS
```

IntelliJ IDEA

1. Open the project folder, mark `src` as *Sources Root* (JDK 17).
2. Run `com.example.app.Main` .
3. Pass program arguments (`ROAD WINDOWS` , etc.) via *Run/Debug Configurations* → *Program arguments*, or just run it and type the values when prompted.

Supported input values

Parameter	Accepted values (case-insensitive)
Delivery mode	ROAD , SEA
UI platform	WINDOWS , MACOS

Values can be supplied as two command-line arguments (`Main <mode> <platform>`) or typed at the interactive prompts. Any other value, or an empty/missing value, is rejected with a clear message and the program stops without performing a partial delivery or building a partial UI.

Sample run

```
$ java -cp out com.example.app.Main ROAD WINDOWS
Delivery mode: ROAD
UI platform: WINDOWS
Rendering Windows button.
Rendering Windows checkbox.
Truck delivers laboratory equipment to Aktau warehouse by road.
```

Invalid-input examples

```
$ java -cp out com.example.app.Main AIR WINDOWS
Unsupported delivery mode: 'AIR'. Supported values: ROAD, SEA.
```

```
$ java -cp out com.example.app.Main ROAD LINUX
Delivery mode: ROAD
Unsupported UI platform: 'LINUX'. Supported values: WINDOWS, MACOS.
```

```
$ java -cp out com.example.app.Main
Delivery mode (ROAD/SEA):
No delivery mode was provided. Supported values: ROAD, SEA.
```